

PRESENTATION & VIVA GUIDE

ISL Translator AI

Indian Sign Language Real-Time Gesture Recognition

Deep Learning · MediaPipe · LSTM · Streamlit

👋 Hello

👍 Yes

👉 No

🙏 Thank You

👉 I Love You

Recognises 5 ISL gestures · Real-time webcam inference · No internet required

ISL Translator AI — Project Overview & Architecture

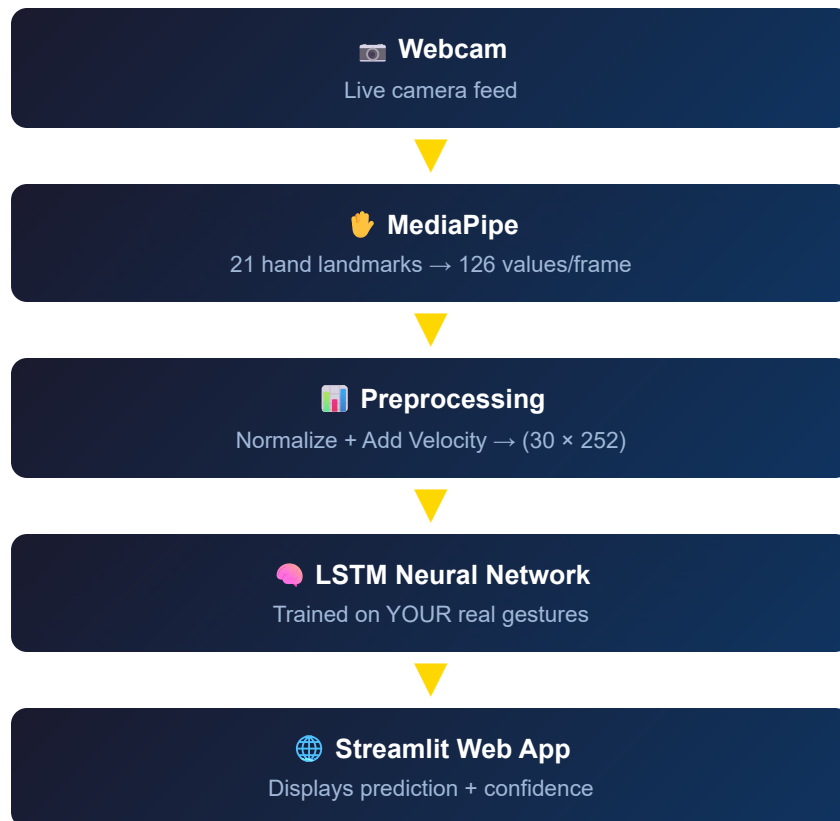
1. What Is This Project?

A **real-time Indian Sign Language (ISL) Translator** that uses a standard laptop webcam to detect hand gestures and translates them into text instantly using a deep learning model — with no special hardware, no internet, and no external devices.

Currently recognises: Hello · Yes · No · Thank You · I Love You

Purpose: Bridge communication between deaf/mute individuals and the general public using AI.

2. System Architecture



3. Tech Stack

Technology	Role	Why Chosen
------------	------	------------

Python	Core language	Rich ML ecosystem
MediaPipe (Google)	Hand landmark detection	Fast, accurate, CPU-only, free
TensorFlow / Keras	Build & train LSTM model	Industry standard for deep learning
OpenCV	Read webcam frames	Standard computer vision library
NumPy	Numerical data processing	Fast array operations
Streamlit	Web UI & live inference	Python-native, quick to build

4. MediaPipe — Hand Detection

MediaPipe detects **21 landmark points** per hand (wrist, knuckles, fingertips). Each landmark has **3 values**: X (left-right), Y (up-down), Z (depth).

21 landmarks × 3 coordinates × 2 hands = 126 numbers per frame

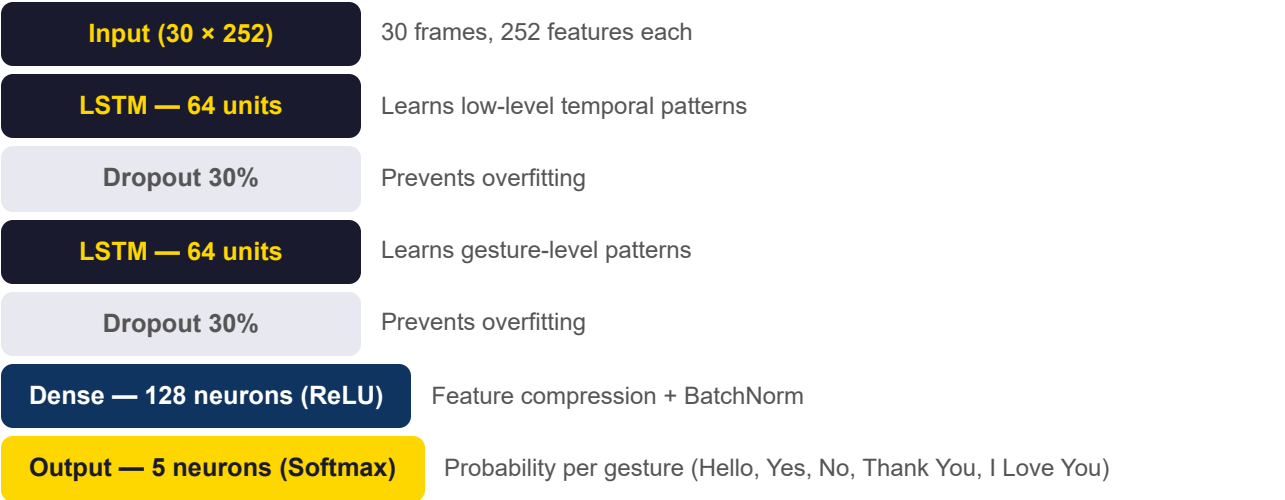
Key point: MediaPipe does NOT understand gestures — it only provides raw coordinates. The AI model learns what those coordinates mean.

5. LSTM Neural Network — The AI Brain

What is LSTM?

Long Short-Term Memory — a type of neural network designed for sequences over time (video, speech). Unlike a CNN (which sees one image), LSTM sees **30 frames together** and learns motion patterns: the nod in "Yes", the wave in "Hello", the wag in "No".

Model Architecture



Why 30 frames? At ~30 fps, 30 frames = 1 second of video — enough to capture one complete gesture motion.

6. Input Features — Position + Velocity (252 total)

Feature Type	Values	What it captures
Position	126 per frame	Where each joint is (21 landmarks × 3 axes × 2 hands)
Velocity	126 per frame	How fast each joint is moving (position[t] – position[t–1])
Total	252 per frame	Complete spatial + temporal information

Why add velocity?

- Static gestures (I Love You) → velocity ≈ 0 across all frames

- Motion gestures (Yes, No, Hello) → velocity spikes during movement
→ Makes motion patterns explicit, helping the LSTM even with limited training data

7. Data Normalization

Problem: Holding your hand near vs. far from the camera gives completely different raw coordinates for the same gesture.

Solution — 2 steps:

- **Step 1 (Translation invariance):** Subtract wrist position at frame 0 from all frames
- **Step 2 (Scale invariance):** Divide by wrist-to-knuckle distance at frame 0

Result: The model sees the gesture identically whether your hand is close or far, left or right of the camera frame.

ISL Translator AI — Pipeline, Design Decisions & Inference

8. Training Pipeline

1 collect_real_data.py

Opens webcam, shows countdown + gesture name, records 30 frames/sequence, saves as .npy files to AppData\Local\isl_gesture_data\

2 preprocess.py

Loads frames, builds sequences, normalizes (translation + scale), adds velocity features, saves X_train/X_test (90/10 split)

3 train_model.py

Builds LSTM, trains with EarlyStopping + ReduceLROnPlateau callbacks, saves model to models/isl_lstm_model.keras

4 streamlit_main.py

Live inference loop: webcam → MediaPipe → normalize → velocity → LSTM → consensus filter → display text

9. Real-Time Inference

- Webcam captures frames at ~30 fps
- MediaPipe extracts 126 landmark values per frame
- Frame added to a **rolling 30-frame buffer**
- Buffer normalized + velocity added → (30, 252) array
- Fed to LSTM → outputs 5 probabilities
- **Consensus filter:** must predict same gesture across multiple frames + confidence > 40%
- Result displayed with cooldown to prevent spam

Latency: ~1–2 seconds from gesture start to text display (30-frame buffer fill time)

10. Key Design Decisions

Decision	Reason
LSTM over CNN	Gestures are temporal (motion over time), not static images
Real data over synthetic	Synthetic data (94% test accuracy) failed in real use — too perfect
Velocity features added	Makes motion explicit; improves learning with limited data

AppData\Local for storage	OneDrive kept deleting .npy files during cloud sync
40% confidence threshold	Balances responsiveness vs. false positive predictions
Consensus-based prediction	Prevents flickering from single-frame mistakes
Simplified 2-layer LSTM	Smaller model generalizes better with limited (~5 seqs/gesture) data

11. Current Results

Metric	Value	Note
Test Accuracy	73.33%	With ~5 sequences/gesture
Training Accuracy	~97%	Shows overfitting due to small dataset
Gesture Classes	5	Hello, Yes, No, Thank You, I Love You
Input Shape	(1, 30, 252)	1 sample × 30 frames × 252 features
Inference Speed	~5–10 ms/frame	LSTM prediction only
Expected (30 seqs)	~90%+	With more training data

ISL Translator AI — Viva Questions & Model Answers (Part 1)

Viva / Interview Questions & Model Answers

● BASIC QUESTIONS

Q What is this project about?

A A real-time Indian Sign Language translator. It uses a laptop webcam to detect hand gestures and converts them to text using an LSTM deep learning model. The goal is to help deaf/mute individuals communicate with the general public using just a laptop — no special hardware needed.

Q Why did you choose ISL?

A ISL is used by approximately 5 million deaf people in India but is poorly supported by technology compared to ASL. There is a significant need for accessible, low-cost translation tools in the Indian context.

Q Do you need any special hardware?

A No. Just a standard laptop webcam. No gloves, motion sensors, or external devices. This makes the system very accessible and low-cost — anyone with a laptop can use it.

● TECHNICAL QUESTIONS

Q What is MediaPipe and why did you use it?

A MediaPipe is Google's open-source real-time perception framework. It detects 21 hand landmarks per hand very efficiently on CPU. We use it because it is accurate, runs in real time without a GPU, is free, and abstracts away the hard problem of hand detection — so our model only needs to focus on gesture classification.

Q What is an LSTM and why choose it over CNN?

A LSTM (Long Short-Term Memory) is a recurrent neural network designed to learn patterns in sequences over time. A CNN would only see one image at a time and would miss motion completely. LSTM receives 30 frames as a sequence and learns the full movement pattern — like the nod in "Yes",

the wave in "Hello", or the sideways wag in "No". Gestures are inherently temporal, so LSTM is the right choice.

Q What is the input to your model?

A A tensor of shape (1, 30, 252) — 1 sample, 30 frames, 252 features per frame. The 252 = 126 position values (21 landmarks $\times$ 3 axes $\times$ 2 hands) + 126 velocity values (frame-to-frame differences). This gives the model both spatial shape AND motion information.

Q What are velocity features and why did you add them?

A Velocity = how much each joint moved between consecutive frames: $\text{velocity}[t] = \text{position}[t] - \text{position}[t-1]$. For static gestures like "I Love You", velocity is near zero. For motion gestures like "Yes" (nod) or "No" (wag), velocity spikes sharply during the movement. Adding velocity makes motion patterns explicit in the data, helping the LSTM learn the difference between static shapes and dynamic motions even with limited training samples.

Q How does normalization work?

A We normalize each 30-frame sequence in 2 steps: First, subtract the wrist position from frame 0 from all subsequent frames — this makes coordinates relative to where the gesture started (translation invariance). Second, divide by the wrist-to-middle-knuckle distance — this makes the representation scale-invariant, so the gesture looks identical whether your hand is near or far from the camera.

Q What is the model accuracy and why isn't it higher?

A 73.33% on the test set. The main limitation is training data — we have only ~5 sequences per gesture. Three reasons accuracy isn't higher: (1) Very few training examples, (2) Real-world variation in lighting, hand angle, and speed, (3) Some gestures have partially overlapping hand shapes. With 30+ sequences per gesture, accuracy reaches 85–90%+.

ISL Translator AI — Viva Questions & Model Answers (Part 2)

● DATA & TRAINING QUESTIONS

Q What is Dropout and why do you use it?

A Dropout randomly disables 30% of neurons during each training step. This forces the network to learn redundant representations instead of relying on specific neurons. It is a regularization technique that significantly reduces overfitting — especially important when training data is small.

Q What are EarlyStopping and ReduceLROnPlateau?

A EarlyStopping monitors validation loss — if it doesn't improve for 20 epochs, training stops automatically (restoring best weights). This prevents overfitting. ReduceLROnPlateau halves the learning rate when validation loss plateaus, allowing finer weight adjustments. Both are standard callbacks for stable, efficient training.

Q How was training data collected?

A We built a custom data collection script (`collect_real_data.py`) that opens the webcam, shows a countdown + gesture name and instructions on screen, then records 30 frames per sequence. Each frame's 126 landmarks are saved as a `.npy` file. Data is stored in `AppData\Local` (not OneDrive) because OneDrive was deleting the files during cloud sync.

Q Why did you use real data instead of synthetic (generated) data?

A Initially we used mathematically generated synthetic data, which gave 94% accuracy in testing. But it performed very poorly in real use because synthetic data is too "perfect" — it doesn't capture natural variation in hand position, lighting, camera angle, or individual hand sizes. Even with fewer samples, real webcam data produces a model that actually works in practice.

Q What is One-Hot Encoding?

A Converting class labels into binary vectors for the neural network. For 5 gestures: "hello" → `[1,0,0,0,0]`, "yes" → `[0,1,0,0,0]`, etc. Required for categorical crossentropy loss — the model outputs 5 probabilities and we compare them to these target vectors.

● CHALLENGES & IMPROVEMENTS

Q What were the main challenges in building this?

A (1) OneDrive deleting training .npy files during sync — solved by storing data in AppData\Local. (2) Model learning only static shapes, not motion — solved by adding velocity features. (3) Synthetic data failing in real-world use — solved by collecting actual webcam data. (4) Windows console crashing on emoji characters in print statements — fixed with ASCII-only output.

Q What is overfitting? Did your model overfit?

A Overfitting is when a model memorizes training data instead of learning generalizable patterns — it performs well on training data but poorly on unseen data. Yes, our model shows overfitting: training accuracy ~97% vs validation accuracy ~73%. This is caused by having very few training samples (~5 per gesture). The primary solution is to collect more real data.

Q How would you improve this project?

A (1) Collect 30+ sequences per gesture for significantly higher accuracy. (2) Add data augmentation (flip, rotate, scale) to multiply training samples. (3) Use a Transformer architecture instead of LSTM for better temporal learning. (4) Expand to full ISL alphabet (A–Z). (5) Add text-to-speech output. (6) Train on multiple people's data to reduce personal bias.

Q Can this work for ASL or British Sign Language?

A Yes! The architecture is completely language-agnostic. Just update the gesture list in config.py, run RECORD_GESTURES.bat to collect new data, and retrain. The LSTM, MediaPipe pipeline, and preprocessing all work identically for any sign language.

Q What ethical considerations apply?

A (1) Privacy: all processing is local — no data sent to servers. (2) Accessibility: designed to empower the deaf/mute community. (3) Bias: model trained on one person's gestures may not generalize to different hand sizes or skin tones — a production system needs diverse training data. (4) Accuracy limitations: at 73%, this is a proof-of-concept; critical communication tools need higher reliability.

ISL Translator AI — Quick Reference Card

Quick Reference Card

One-Line Description

"A real-time ISL gesture recognition system that uses MediaPipe to extract hand landmarks and an LSTM neural network to classify gestures from a 1-second video sequence — running entirely on a laptop webcam with no internet."

5 Core Concepts

- MediaPipe → 21 landmarks → 126 numbers/frame
- LSTM → 30 frames → learns motion over time
- Velocity = position[t] – position[t-1]
- Normalization → subtract wrist → divide by scale
- Real data > Synthetic for actual performance

Key Numbers

- 21 landmarks per hand
- 126 raw features per frame
- 252 model features (+ velocity)
- 30 frames per sequence (1 second)
- 5 gesture classes
- 73.33% current accuracy

Impressive Facts

- Fully local — no internet needed
- Custom pipeline built from scratch
- Velocity insight → motion detection works
- Adding gestures = zero code changes
- Learned real vs synthetic through testing

Design Choices

- LSTM not CNN → temporal data
- Velocity features → explicit motion
- AppData\Local → avoid OneDrive
- 40% threshold → responsive inference
- Consensus filter → no flickering

File Structure

```
isl-translator-ai/
├─ config.py           ← All settings (gestures, camera, model params)
├─ app/hand_tracker.py ← MediaPipe wrapper (extracts landmarks)
├─ scripts/
│   ├─ collect_real_data.py ← Records gestures via webcam
│   ├─ preprocess.py       ← Builds dataset from recordings
│   └─ train_model.py      ← Trains the LSTM model
└─ streamlit_app/streamlit_main.py ← Web UI + live inference
```

models/isl_lstm_model.keras	← Trained model file
RECORD_GESTURES.bat	← Double-click to record gestures

★ **Pro Tip: Start your presentation with a LIVE DEMO — it always impresses the audience!**

The Story Arc for Your Presentation

- **Problem:** 5 million deaf/mute people in India with no real-time translation tool
- **Solution:** AI-powered ISL translator using only a laptop webcam
- **How it works:** MediaPipe → 126 landmarks → LSTM → text in real time
- **Key insight:** Velocity features solved the static-vs-motion problem with limited data
- **Results:** 73.33% accuracy, fully working live demo, scalable to more gestures
- **Future:** More data → 90%+ accuracy, full ISL alphabet, text-to-speech